

LAPORAN PRATIKUM PEKAN 6

STRUKTUR DATA

disusun Oleh:

ALIF MUHAMMAD IHSAN

NIM 2511531004

Dosen Pengampu : Dr.Wahyudi S.T, M.T

Asisten : M Galid Avero



DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

PADANG

2025

KATA PENGANTAR

Puji syukur ke hadirat Allah SWT atas segala rahmat dan karunia-Nya sehingga penulis dapat menyelesaikan laporan praktikum **Struktur Data** ini dengan baik dan tepat waktu. Laporan ini disusun sebagai salah satu bentuk pertanggungjawaban atas kegiatan praktikum yang telah dilaksanakan, sekaligus sebagai sarana untuk memperdalam pemahaman mengenai konsep dasar struktur data, implementasi algoritma, serta penerapan logika pemrograman dalam kehidupan nyata.

Dalam penyusunan laporan ini, penulis menyadari masih terdapat banyak kekurangan, baik dari segi isi maupun penyajian. Oleh karena itu, penulis sangat mengharapkan kritik dan saran yang membangun demi perbaikan di masa mendatang.

Ucapan terima kasih penulis sampaikan kepada:

- Dosen pengampu mata kuliah **Struktur Data** yang telah memberikan arahan dan bimbingan.
- Asisten praktikum yang senantiasa membantu dalam memahami materi dan menyelesaikan permasalahan teknis.
- Rekan-rekan mahasiswa yang turut berdiskusi dan bekerja sama selama kegiatan praktikum berlangsung.

Akhir kata, semoga laporan ini dapat memberikan manfaat, baik bagi penulis sendiri maupun bagi pembaca yang ingin mempelajari lebih lanjut mengenai struktur data.

Daftar Isi

KATA PENGANTAR	i
BAB I	3
PENDAHULUAN.....	3
1.1 Latar Belakang	3
1.2 Tujuan	3
1.3 Manfaat Praktikum	4
BAB II	5
PEMBAHASAN	5
2.1 Kode Program	3
2.2 Langkah Kerja	11
2.3 Analisis Hasil	19
BAB III.....	19
PENUTUP.....	19
3.1 Kesimpulan	19
3.2 Saran	19
DAFTAR PUSTAKA.....	20

BAB I

PENDAHULUAN

1.1 Latar Belakang

Struktur data merupakan salah satu aspek fundamental dalam ilmu komputer yang berperan penting dalam pengelolaan dan pengorganisasian data. Dengan adanya struktur data, proses penyimpanan, pengambilan, serta manipulasi data dapat dilakukan secara lebih efisien dan terstruktur. Pemahaman mengenai struktur data menjadi dasar bagi pengembangan algoritma yang optimal, sehingga sangat berpengaruh terhadap performa suatu program maupun sistem.

Melalui praktikum struktur data, mahasiswa tidak hanya mempelajari konsep teoritis, tetapi juga mengimplementasikan langsung berbagai bentuk struktur data seperti array, linked list, stack, queue, dan tree dalam bahasa pemrograman. Praktikum ini bertujuan untuk melatih keterampilan analisis logika, pemecahan masalah, serta kemampuan debugging yang diperlukan dalam pengembangan perangkat lunak.

Selain itu, praktikum ini juga memberikan pengalaman nyata dalam menghubungkan teori dengan praktik, sehingga mahasiswa dapat memahami bagaimana suatu algoritma bekerja di balik layar dan bagaimana pemilihan struktur data yang tepat dapat memengaruhi efisiensi program. Dengan demikian, kegiatan praktikum struktur data menjadi sarana penting untuk membangun fondasi keilmuan yang kuat dalam bidang informatika.

1.2 Tujuan

1. Memahami konsep dasar struktur data dan penerapannya dalam pemrograman.
2. Melatih keterampilan dalam mengimplementasikan berbagai jenis struktur data seperti array, linked list, stack, queue, dan tree menggunakan bahasa pemrograman.
3. Mengembangkan kemampuan analisis logika dan pemecahan masalah melalui studi kasus yang diberikan dalam praktikum.
4. Melatih keterampilan debugging serta troubleshooting program agar dapat menemukan dan memperbaiki kesalahan secara sistematis.

1.3 Manfaat Praktikum

1. Memberikan pengalaman langsung dalam mengimplementasikan struktur data sehingga mahasiswa lebih mudah dalam memahami konsep yang abstrak
2. Meningkatkan keterampilan pemograman, khususnya dalam efesiensi algoritma dan pengelolaan data.
3. Menjadi bekal penting untuk mata kuliah lanjutan maupun proyek pengembangan perangkat lunak di masa depan.
4. Membantu mahasiswa memahami pentingnya pemilihan struktur data yang tepat dalam meningkatkan peforma program.

BAB II

PEMBAHASAN

2.1 Kode Program

a. hapusDDL_2511531004

```
package Pekan6_2511531004;

public class hapusDDL_2511531004 {
    // fungsi untuk menghapus node awal
    public static NodeDLL_2511531004 deleteHead_2511531004
(NodeDLL_2511531004 head_1004) {
        if (head_1004 == null) {
            return null; }

        head_1004 = head_1004.next_1004;
        if (head_1004 != null) {
            head_1004.prev_1004 = null; }

        return head_1004;
    }

    // fungsi untuk menghapus di akhir
    public static NodeDLL_2511531004 deleteLast_2511531004
(NodeDLL_2511531004 head_1004) {
        if (head_1004 == null ) {
            return null; }

        if (head_1004.next_1004 == null ) {
            return null; }
        NodeDLL_2511531004 curr = head_1004;

        while (curr.next_1004 != null ) {
            curr = curr.next_1004;
        }

        // updating pointer previous node
        if (curr.prev_1004 != null ) {
            curr.prev_1004.next_1004 = null ; }
        return head_1004;
    }

    // fungsi untuk menghapus node posisi tertentu
    public static NodeDLL_2511531004 delPos_2511531004
(NodeDLL_2511531004 head_1004 , int pos_1004) {
```

```

        // jika dll kosong
        if (head_1004 == null ) {
            return head_1004;
        }
        NodeDLL_2511531004 curr = head_1004;
        // telusuri sampai ke node yang akan dihapus
        for (int i = 1; curr != null && i < pos_1004; ++ i) {
            curr = curr.next_1004;
        }
        // jika posisi tidak ditemukan
        if (curr == null ) {
            return head_1004;
        }

        //update poinnter
        if (curr.prev_1004 != null ) {
            curr.prev_1004.next_1004 = curr.next_1004;
        }

        if ( curr.next_1004 != null ) {
            curr.next_1004.prev_1004 = curr.prev_1004 ;
        }

        // jika yang dihapus head
        if (head_1004 == curr ) {
            head_1004 = curr.next_1004;
        }
        return head_1004;
    }

    // fungsi untuk mencetak dll
    public static void printList_2511531004 (NodeDLL_2511531004
head_1004 ) {
        NodeDLL_2511531004 curr = head_1004;
        while ( curr != null ) {
            System.out.print(curr.data_1004 + " ");
            curr = curr.next_1004;
        }
        System.out.println();
    }

    public static void main(String[] args) {
        // buat sebauh dll
        NodeDLL_2511531004 head_1004 = new NodeDLL_2511531004 (1);
        head_1004.next_1004 = new NodeDLL_2511531004 (2);
        head_1004 .next_1004.prev_1004 = head_1004;
        head_1004.next_1004.next_1004 = new NodeDLL_2511531004 (3);
        head_1004.next_1004.next_1004.prev_1004 =
head_1004.next_1004;
        head_1004.next_1004.next_1004.next_1004 = new
NodeDLL_2511531004 (4) ;
        head_1004.next_1004.next_1004.next_1004.prev_1004 =
head_1004.next_1004.next_1004;
        head_1004.next_1004.next_1004.next_1004.next_1004 = new
NodeDLL_2511531004 (5);
    }

```

```

        head_1004.next_1004.next_1004.next_1004.next_1004.prev_1004
= head_1004.next_1004.next_1004.next_1004;

        System.out.print("DLL diawali : ");
        printList_2511531004 (head_1004);

        System.out.print("setelah head dihapus : ");
        head_1004 = deleteHead_2511531004(head_1004);
        printList_2511531004 (head_1004);

        System.out.print("Setelah node terakhir di hapus : ");
        head_1004 = deleteLast_2511531004 (head_1004);
        printList_2511531004 (head_1004);

        System.out.print("Menghapus node ke 2 : ");
        head_1004 = delPos_2511531004 (head_1004, 2);
        printList_2511531004 (head_1004);

    }

}

```

b. insertDDL_2511531004

```

package Pekan6_2511531004;

public class InsertDLL_2511531004 {
    // menambahkan node di awal dll

    static NodeDLL_2511531004 insertBegin(NodeDLL_2511531004
head_1004, int data_1004) {
        // buat node baru
        NodeDLL_2511531004 new_node = new NodeDLL_2511531004
(data_1004);
        // jadikan pointer nextnya head
        new_node.next_1004 = head_1004;
        // jadikan pointer prev head ke new node
        if (head_1004 != null) {
            head_1004.prev_1004 = new_node;
        }
        return new_node;
    }

    // fungsi untuk menambahkan node di akhir
    public static NodeDLL_2511531004 insertEnd(NodeDLL_2511531004
head_1004 , int newData_1004) {
        // buat node baru
        NodeDLL_2511531004 newNode = new NodeDLL_2511531004
(newData_1004) ;
    }
}

```



```

        // jika dll null di jadikan head
        if (head_1004 == null) {
            head_1004 = newNode;
        }

        else {
            NodeDLL_2511531004 curr = head_1004;
            while (curr.next_1004 != null) {
                curr = curr.next_1004;
            }
            curr.next_1004 = newNode;
            newNode.prev_1004 = curr;
        }
        return head_1004;
    }

    // fungsi untuk menambahkan node di posisi tertentu
    public static NodeDLL_2511531004 insertAtPosition
(NodeDLL_2511531004 head_1004, int pos_1004, int new_data_1004) {
        // buat node baru
        NodeDLL_2511531004 new_node = new NodeDLL_2511531004
(new_data_1004);
        if ( pos_1004 == 1) {
            new_node.next_1004 = head_1004;
            if (head_1004 != null ) {
                head_1004.prev_1004 = new_node; }
            head_1004 = new_node;
            return head_1004; }

        NodeDLL_2511531004 curr = head_1004;

        for (int i = 1; i < pos_1004 - 1 && curr != null; ++i) {
            curr = curr.next_1004;
        }

        if (curr == null) {
            System.out.println("Posisi tidak ada");
            return head_1004; }
        new_node.prev_1004 = curr;
        new_node.next_1004 = curr.next_1004;
        curr.next_1004 = new_node;

        if (new_node.next_1004 != null) {
            new_node.next_1004.prev_1004 = new_node; }

        return head_1004;
    }

    public static void printList_2511531004 (NodeDLL_2511531004
head_1004) {
        NodeDLL_2511531004 curr = head_1004;
        while (curr != null) {
            System.out.print(curr.data_1004 + " <---> ");
            curr = curr.next_1004;
        }
    }

```

```

    }

    System.out.println();
}

public static void main(String[] args) {
    // membuat dll
    NodeDLL_2511531004 head_1004 = new NodeDLL_2511531004 (2);
    head_1004.next_1004 = new NodeDLL_2511531004 (3);
    head_1004.next_1004.prev_1004 = head_1004;
    head_1004.next_1004.next_1004 = new NodeDLL_2511531004 (5);
    head_1004.next_1004.next_1004.prev_1004 = head_1004;

    // cetak DLL awal
    System.out.print("DLL AWAL : ");
    printList_2511531004(head_1004);

    // tambah 1 di awal
    head_1004 = insertBegin(head_1004,1);
    System.out.print("Simpul ditambah 1 di awal : ");
    printList_2511531004(head_1004);

    // tambah 6 diakhir
    System.out.print("simpul 6 di tambah di akhir : ");
    int data_1004 = 6;
    head_1004 = insertEnd (head_1004, data_1004);
    printList_2511531004 (head_1004);

    // menambah node 4 di posisi 4
    System.out.print("tambah node 4 di posisi 4 : ");

    int data2_1004 = 4;
    int pos_1004 = 4;
    head_1004 = insertAtPosition(head_1004 , pos_1004,
data2_1004);
    printList_2511531004 (head_1004);

}
}

```

c. NodeDDL_2511531004

```

package Pekan6_2511531004;

public class NodeDLL_2511531004 {
    // mendefinisikan kelas node
    int data_1004 ;
    NodeDLL_2511531004 next_1004; // pointer ke node selanjutnya
    NodeDLL_2511531004 prev_1004; // pointer ke node sebelumnya

    // konstruktor

```

```

        public NodeDLL_2511531004 (int data_1004) {
            this.data_1004 = data_1004;
            this.next_1004 = null;
            this.prev_1004 = null;
        }
    }
}

```

d. PenelusuranDDL_2511531004

```

package Pekan6_2511531004;

public class penelusuranDLL_2511531004 {

    // fungsi penelusuran maju
    static void forwardTraversal_2511531004 (NodeDLL_2511531004
head_1004 ) {
        // memulai penelusuran dari head
        NodeDLL_2511531004 curr = head_1004;
        // lanjutkan sampai akhir
        while ( curr != null) {
            // print data
            System.out.print(curr.data_1004 + " <----> ");
            // pindah ke node berikutnya
            curr = curr.next_1004;
        }
        System.out.println();
    }

    // fungsi untuk penelusuran mundur
    static void backwardTraversal_2511531004 (NodeDLL_2511531004
tail_1004) {
        // mulai dari akhir
        NodeDLL_2511531004 curr = tail_1004;
        // lanjut sampai head
        while (curr != null) {
            // cetak data
            System.out.print(curr.data_1004 + " <----> ");
            // pindah ke node sebelumnya
            curr = curr.prev_1004;
        }
        // cetak spasi
        System.out.println();
    }

    public static void main(String[] args ) {
        // cetak dll
        NodeDLL_2511531004 head_1004 = new NodeDLL_2511531004 (1);
    }
}

```

```

NodeDLL_2511531004 second_1004 = new NodeDLL_2511531004
(2);
NodeDLL_2511531004 third_1004 = new NodeDLL_2511531004 (3);

head_1004.next_1004 = second_1004;
second_1004.prev_1004 = head_1004;
second_1004.next_1004 = third_1004;
third_1004.prev_1004 = second_1004;

System.out.println("Penelusuran maju : ");
forwardTraversal_2511531004(head_1004);

System.out.println("Penelusuran mundur : ");
backwardTraversal_2511531004 (third_1004);
}

}

```

2.2 Langkah Kerja

a. hapusDDL_2511531004

1. Membuat DLL Awal

- Node dibuat manual: $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5$
- Setiap node punya pointer next_1004 (ke kanan) dan prev_1004 (ke kiri).
- Hasil awal:
- DLL diawali : 1 2 3 4 5

2. Fungsi deleteHead_2511531004

- Mengecek apakah list kosong (head == null). Jika ya, return null.
- Jika tidak kosong:
 - head digeser ke node berikutnya (head = head.next_1004).
 - Pointer prev_1004 dari node baru di-set null.
- Hasil:
- setelah head dihapus : 2 3 4 5

3. Fungsi deleteLast_2511531004

- Mengecek apakah list kosong atau hanya 1 node. Jika ya, return null.
- Traversal dari head sampai node terakhir (curr).
- Node terakhir dihapus dengan cara:
 - `curr.prev_1004.next_1004 = null`.
- Hasil:
- Setelah node terakhir dihapus : 2 3 4

4. Fungsi delPos_2511531004

- Mengecek apakah list kosong. Jika ya, return head.
- Traversal sampai posisi yang diminta (`pos_1004`).
- Jika posisi tidak ditemukan (`curr == null`), return head.
- Jika ditemukan:
 - Update pointer `prev_1004` dan `next_1004` agar node dilewati.
 - Jika node yang dihapus adalah head, maka head digeser ke node berikutnya.
- Hasil (hapus node ke-2):
- Menghapus node ke 2 : 2 4

5. Fungsi printList_2511531004

- Traversal dari head ke null.
- Cetak data_1004 tiap node.

b. insertDDL_2511531004

1. Membuat DLL Awal

- Node dibuat manual: $2 \rightarrow 3 \rightarrow 5$
- Setiap node punya pointer `next_1004` (ke kanan) dan `prev_1004` (ke kiri).

- Cetak awal:
- DLL AWAL : 2 <---> 3 <---> 5

2. Fungsi insertBegin

- Membuat node baru dengan data 1.
- `new_node.next_1004 = head_1004` → node baru menunjuk ke head lama.
- `head_1004.prev_1004 = new_node` → head lama menunjuk balik ke node baru.
- head diperbarui ke node baru.
- Hasil:
- Simpul ditambah 1 di awal : 1 <---> 2 <---> 3 <---> 5

3. Fungsi insertEnd

- Membuat node baru dengan data 6.
- Traversal dari head sampai node terakhir (5).
- `curr.next_1004 = newNode` → node terakhir menunjuk ke node baru.
- `newNode.prev_1004 = curr` → node baru menunjuk balik ke node terakhir.
- Hasil:
- simpul 6 ditambah di akhir : 1 <---> 2 <---> 3 <---> 5 <---> 6

4. Fungsi insertAtPosition

- Membuat node baru dengan data 4.
- Posisi yang diminta = 4.
- Traversal sampai node ke-3 (3).
- `new_node.prev_1004 = curr` → node baru menunjuk ke node 3.
- `new_node.next_1004 = curr.next_1004` → node baru menunjuk ke node 5.
- Update pointer:

- `curr.next_1004 = new_node` → node 3 menunjuk ke node 4.
- `new_node.next_1004.prev_1004 = new_node` → node 5 menunjuk balik ke node 4.
- Hasil:
- tambah node 4 di posisi 4 : 1 <---> 2 <---> 3 <---> 4 <---> 5 <---> 6

c. PenelusuranDDL_2511531004

1. Membuat DLL Awal

- Node dibuat manual:
 - `head = 1`
 - `second = 2`
 - `third = 3`
- Pointer dihubungkan:
 - `head.next = second`
 - `second.prev = head`
 - `second.next = third`
 - `third.prev = second`
- Struktur awal:
- 1 <-----> 2 <-----> 3

2. Fungsi forwardTraversal_2511531004

- Mulai dari **head** (1).
- Cetak data node, lalu pindah ke next.
- Proses berulang sampai `curr == null`.
- Hasil cetakan:
- Penelusuran maju :

- 1 <----> 2 <----> 3 <---->

3. Fungsi backwardTraversal_2511531004

- Mulai dari **tail** (3).
- Cetak data node, lalu pindah ke prev.
- Proses berulang sampai curr == null.
- Hasil cetakan:
- Penelusuran mundur :
- 3 <----> 2 <----> 1 <---->

d. NodeDLL_2511531004

1. Deklarasi Class Node

- Membuat class NodeDLL_2511531004 sebagai **template node** untuk Doubly Linked List (DLL).
- Class ini punya 3 atribut:
 - int data_1004 → menyimpan nilai data.
 - NodeDLL_2511531004 next_1004 → pointer ke node berikutnya.
 - NodeDLL_2511531004 prev_1004 → pointer ke node sebelumnya.

2. Konstruktor Node

- Konstruktor menerima parameter data_1004.
- this.data_1004 = data_1004 → mengisi nilai data.
- this.next_1004 = null → pointer ke node berikutnya default null.
- this.prev_1004 = null → pointer ke node sebelumnya default null.

3. Alur Kerja Node dalam DLL

- Saat node baru dibuat, misalnya:
- `NodeDLL_2511531004 node1 = new NodeDLL_2511531004(10);`

Maka node node1 berisi data 10, dengan `next_1004 = null` dan `prev_1004 = null`.

- Jika ingin menghubungkan node:
- `NodeDLL_2511531004 node2 = new NodeDLL_2511531004(20);`
- `node1.next_1004 = node2;`
- `node2.prev_1004 = node1;`

Maka terbentuk DLL sederhana:

10 <--> 20

BAB III

PENUTUP

3.1 Kesimpulan

1. **NodeDLL_2511531004** menjadi struktur dasar DLL dengan atribut data, next, dan prev sehingga memungkinkan penelusuran dua arah.
2. Program **InsertDLL** berhasil menambahkan node di awal, akhir, dan posisi tertentu dengan memperbarui pointer next dan prev agar list tetap konsisten.
3. Program **hapusDLL** mampu menghapus node di awal, akhir, maupun posisi tertentu dengan menjaga keterhubungan antar node.
4. Program **penelusuranDLL** menunjukkan kelebihan DLL yaitu bisa ditelusuri **maju (head → tail)** maupun **mundur (tail → head)**.
5. Secara keseluruhan, implementasi DLL ini sudah mencakup operasi dasar yang penting: **insert, delete, dan traversal**, sehingga DLL dapat digunakan sebagai fondasi untuk struktur data yang lebih kompleks.

3.2 Saran

1. **Tambahkan validasi input** agar program lebih aman, misalnya saat posisi yang diminta untuk insert/delete melebihi panjang list.
2. **Buat fungsi pencarian (search)** untuk menemukan node berdasarkan nilai tertentu, sehingga DLL lebih fleksibel digunakan.
3. **Implementasikan update node** (mengubah data pada posisi tertentu) agar operasi DLL lebih lengkap.
4. **Gunakan kelas wrapper** (misalnya DoublyLinkedList) untuk menyatukan semua operasi (insert, delete, traversal) agar lebih terstruktur dan mudah dipakai ulang.
5. **Uji dengan dataset lebih besar** untuk memastikan pointer tetap konsisten dan tidak terjadi error saat banyak node ditambahkan atau dihapus.

DAFTAR PUSTAKA

Oracle. (2024). *Java™ Platform, Standard Edition Documentation*. Oracle.
<https://docs.oracle.com/javase/8/docs/api/> ([docs.oracle.com in Bing](#))

Deitel, P. J., & Deitel, H. M. (2017). *Java: How to Program (Early Objects)* (11th ed.). Pearson Education.

Liang, Y. D. (2019). *Introduction to Java Programming and Data Structures* (11th ed.). Pearson.

Wirth, N. (2006). *Algorithms and Data Structures*. Springer. (Referensi umum untuk konsep struktur data seperti ArrayList).

GeeksforGeeks. (2025). *Java ArrayList – Methods and Examples*.
<https://www.geeksforgeeks.org/arraylist-in-java/> ([geeksforgeeks.org in Bing](#))

TutorialsPoint. (2025). *Java – Object-Oriented Programming Concepts*.
https://www.tutorialspoint.com/java/java_object_classes.htm ([tutorialspoint.com in Bing](#))

Tugas Struktur Data Pekan 6

a. Lagu_2511531004

```
• package Pekan6_2511531004;
•
• public class Lagu_2511531004 {
•     private String judul_1004;
•     private String penyanyi_1004;
•     Lagu_2511531004 next_1004;
•     Lagu_2511531004 prev_1004;
•
•     // Constructor
•     public Lagu_2511531004(String judul_1004, String
penyanyi_1004) {
•         this.judul_1004 = judul_1004;
•         this.penyanyi_1004 = penyanyi_1004;
•         this.next_1004 = null;
•         this.prev_1004 = null;
•     }
•
•     // Getter
•     public String getJudul_1004() { return judul_1004; }
•     public String getPenyanyi_1004() { return penyanyi_1004; }
•
•     // Setter
•     public void setJudul_1004(String judul_1004) {
this.judul_1004 = judul_1004; }
•     public void setPenyanyi_1004(String penyanyi_1004) {
this.penyanyi_1004 = penyanyi_1004; }
• }
```

b. Musik_2511531004

```
package Pekan6_2511531004;

public class Musik_2511531004 {
    private Lagu_2511531004 head_1004;
    private Lagu_2511531004 tail_1004;

    public Musik_2511531004() {
        head_1004 = null;
        tail_1004 = null;
    }

    // 1. Tambah Lagu di akhir
    public void tambahLagu_1004(String judul, String penyanyi) {
        Lagu_2511531004 baru = new Lagu_2511531004(judul, penyanyi);
        if (head_1004 == null) {
            head_1004 = tail_1004 = baru;
        } else {
            tail_1004.next_1004 = baru;
            baru.prev_1004 = tail_1004;
        }
    }
}
```

```

        tail_1004 = baru;
    }
    System.out.println("Lagu berhasil ditambahkan!");
}

// 2. Hapus Lagu Awal
public void hapusLaguAwal_1004() {
    if (head_1004 == null) {
        System.out.println("Playlist kosong!");
        return;
    }
    System.out.println("Menghapus: " + head_1004.getJudul_1004());
    head_1004 = head_1004.next_1004;
    if (head_1004 != null) {
        head_1004.prev_1004 = null;
    } else {
        tail_1004 = null;
    }
}

// 3. Tampil Maju
public void tampilMaju_1004() {
    if (head_1004 == null) {
        System.out.println("Playlist kosong!");
        return;
    }
    Lagu_2511531004 temp = head_1004;
    while (temp != null) {
        System.out.println(temp.getJudul_1004() + " - " +
temp.getPenyanyi_1004());
        temp = temp.next_1004;
    }
}

// 4. Tampil Mundur
public void tampilMundur_1004() {
    if (tail_1004 == null) {
        System.out.println("Playlist kosong!");
        return;
    }
    Lagu_2511531004 temp = tail_1004;
    while (temp != null) {
        System.out.println(temp.getJudul_1004() + " - " +
temp.getPenyanyi_1004());
        temp = temp.prev_1004;
    }
}

// 5. Cari Lagu
public void cariLagu_1004(String judul) {
    if (head_1004 == null) {
        System.out.println("Playlist kosong!");
        return;
    }
    Lagu_2511531004 temp = head_1004;
    boolean ketemu = false;
    while (temp != null) {

```

```

        if (temp.getJudul_1004().equalsIgnoreCase(judul)) {
            System.out.println("Lagu ditemukan: " +
temp.getJudul_1004() + " - " + temp.getPenyanyi_1004());
            ketemu = true;
            break;
        }
        temp = temp.next_1004;
    }
    if (!ketemu) {
        System.out.println("Lagu tidak ditemukan!");
    }
}
}
}

```

c. MusikDriver_2511531004

```

package Pekan6_2511531004;
import java.util.Scanner;
public class MusikDriver_2511531004 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Musik_2511531004 playlist = new Musik_2511531004();
        int pilihan;

        do {
            System.out.println("=== Playlist Musik NIM:
2511531004 ===");
            System.out.println("1. Tambah Lagu");
            System.out.println("2. Hapus Lagu Pertama");
            System.out.println("3. Lihat Playlist (Maju)");
            System.out.println("4. Lihat Playlist (Mundur)");
            System.out.println("5. Cari Lagu");
            System.out.println("6. Keluar");
            System.out.print("Pilihan: ");
            pilihan = sc.nextInt();
            sc.nextLine(); // clear buffer

            switch (pilihan) {
                case 1:
                    System.out.print("Judul: ");
                    String judul = sc.nextLine();
                    System.out.print("Penyanyi: ");
                    String penyanyi = sc.nextLine();
                    playlist.tambahLagu_1004(judul, penyanyi);
                    break;
                case 2:
                    playlist.hapusLaguAwal_1004();
                    break;
                case 3:
                    playlist.tampilMaju_1004();
                    break;
                case 4:
                    playlist.tampilMundur_1004();
                    break;
            }
        } while (pilihan != 6);
    }
}

```

```

        case 5:
            System.out.print("Masukkan judul lagu: ");
            String cari = sc.nextLine();
            playlist.carilagu_1004(cari);
            break;
        case 6:
            System.out.println("Keluar...");
            break;
        default:
            System.out.println("Pilihan tidak valid!");
    }
} while (pilihan != 6);

sc.close();
}
}

```

Output

```

1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari lagu
6. Keluar
Pilihan: 1
Judul: heal the word
Penyanyi: michael jackson
Lagu berhasil ditambahkan!
=== Playlist Musik NIM: 2511531004 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari lagu
6. Keluar
Pilihan: 5
Judul: heal the word - michael jackson
Penyanyi: hindia
Lagu berhasil ditambahkan!
=== Playlist Musik NIM: 2511531004 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari lagu
6. Keluar
Pilihan: 6
Keluar

```